Port.io Demo Cheat Sheet — Walmart Production Readiness

🎯 Port.io Demo — 10-Minute Walkthrough Cheat Sheet

Walmart Production Readiness | TSM/TSE Assignment

URL: https://app.getport.io **Total time:** ~10 minutes | **Sections:** 7 stops

---

⏱ TIME MAP

Time	Stop	What you show
0:00–1:00	Dashboard	The single pane of glass
1:00–2:30	Services Catalog	6 services, relations, scorecard levels
2:30–4:00	Scorecards	Gold/Silver rules explained
4:00–5:30	Self-Service: Snyk Scan	Trigger GitHub Actions from Port
5:30–6:30	Self-Service: Create Jira Issue	Dev creates ticket without leaving Port
6:30–8:00	Integrations	5 live integrations, all healthy
8:00–10:00	Datadog + Wrap-up	Live metrics, the “before vs after” close

---

STOP 1 — Dashboard (1 min)

Where: Left sidebar → “Walmart - Production Readiness”

What to say: > “This is the single pane of glass for Walmart’s platform team. Everything a developer or platform engineer needs — in one view.”

What to point at: 1. **Top-left table** — “6 Walmart production services. You can see on-call owner, open CVEs, change failure rate.” 2. **Top-right pie** — “This is real Snyk data. 3 critical, 3 high, 2 medium CVEs detected from scanning our actual npm dependencies.” 3. **Middle-left table** — “K8s workloads. All 6 are Healthy, all have resource limits, none are privileged, none use ‘latest’ tags. That’s Gold scorecard.” 4. **Middle-right table** — “PagerDuty. Every service has an on-call engineer assigned.

That's Silver scorecard." 5. **Datadog monitors table** — "4 Synthetics health checks from Datadog — checkout, payment, inventory, auth — all showing OK." 6. **Bottom table** — "14 Jira tickets in the KAN project. 6 of them were auto-created when services hit Gold scorecard."

Key line: > "Before Port, this data lived in 5 different tools. Now it's one screen."

STOP 2 — Services Catalog (1:30 min)

Where: Left sidebar → "Services" (under Software Catalog)

What to say: > "Here are the 6 Walmart production microservices. Let me click into checkout-service."

Click: checkout-service

What to point at: 1. **Relations panel** (right side or bottom): - `repository` → port.io (GitHub) - `pager_duty_service` → PD-CHECKOUT - "Every service is connected to its GitHub repo and PagerDuty service automatically."

2. **Scorecard section** (scroll down):
    - Show `ProductionReadinessGithubOcean` scorecard
    - "This is the production readiness scorecard. It evaluates rules automatically every time the entity changes."
3. **Mirror properties:**
    - `pagerduty_oncall` → payments-oncall@walmart.com
    - `open_critical_vulnerabilities` → number from Snyk
    - "These values are pulled live from PagerDuty and Snyk. No manual entry."

Key line: > "A developer can look at this one page and know: is my service production-ready? Who's on-call? Do I have critical CVEs? All without switching tools."

STOP 3 — Scorecards (1:30 min)

Where: Left sidebar → "Scorecards" (or navigate to a K8s Pod entity)

Option A — Show K8s Pod scorecard: - Left sidebar → "K8s Pods" - Click any pod (e.g., `checkout-service-pod-1`) - Show "Pod Production Readiness" scorecard

What to point at: - **Bronze rule:** “Workload is Healthy” → ✅ SUCCESS - **Silver rule:** “All containers have resource limits” → ✅ SUCCESS - **Silver rule:** “No privileged containers” → ✅ SUCCESS - **Gold rule:** “No ‘latest’ image tag” → ✅ SUCCESS

What to say: > “Bronze, Silver, Gold. This is the production readiness ladder. Every rule maps to a real operational risk. ‘Latest’ image tag means you can’t reproduce a deploy. Privileged containers are a security risk. Resource limits prevent noisy-neighbor problems in K8s.”

Option B — Show PagerDuty scorecard: - Left sidebar → “PagerDuty Services” - Click `PD-CHECKOUT` - Show “Production Ready” scorecard → Silver level - Rule: “Has PagerDuty on-call” → ✅ SUCCESS

Key line: > “Gold scorecard means: this service is ready to ship. Not because someone approved it — because it objectively meets every rule. That’s the difference.”

STOP 4 — Self-Service: Snyk Security Scan (1:30 min)

Where: Left sidebar → “Services” → click `checkout-service` → “Actions” tab (top of entity page)

What to say: > “Step 4 in our flow: the developer triggers a Snyk scan directly from Port. No CLI, no CSV email, no waiting.”

Steps to demo: 1. Click **“Snyk Security Scan”** action 2. Set `severity_threshold` = **high** 3. Click **Execute** 4. Show the run log appearing: “Initiating action. Sending to GitHub Actions...”

What to say while it runs: > “This is triggering our `snyk-scan.yml` GitHub Actions workflow. It checks out the code, runs a full Snyk scan, parses the JSON output, and reports the vulnerability count back to Port in real-time. The developer never left Port.”

If it completes during demo: - Show the run log with vulnerability count - “Results are back. Zero critical CVEs. The scorecard updates automatically.”

Key line: > “Before Port: developer runs Snyk locally, emails a CSV to the security team, waits for a response. With Port: one click, results in 2 minutes, automatically linked to the service entity.”

STOP 5 — Self-Service: Create Jira Issue (1 min)

Where: Left sidebar → “Self-Service” (or from any entity → Actions tab)

What to say: > “Step 4 also includes creating Jira tickets. A developer can raise a production readiness issue without leaving Port.”

Steps to demo: 1. Click **“Create an issue”** action 2. Fill in: - **Issue type:** Task - **Title:** `[checkout-service] Pre-deploy readiness check — Gold achieved` 3. Click **Execute** 4. Show: “Status: Success”

What to say: > “That just created KAN-16 in our Jira project. The developer didn’t open Jira, didn’t copy-paste a service name, didn’t fill in a project key. Port handled it.”

Bonus — show the automation: - Go to **Builder** → **Automations** → find `gold_scorecard_achieved` - “This automation fires automatically when any service hits Gold. It creates the Jira ticket without any human action. That’s the 19 automations running 24/7 in the background.”

STOP 6 — Integrations (1:30 min)

Where: Left sidebar → “Builder” → “Integrations” (or Settings → Integrations)

What to show: Point to each integration and its status:

Integration	Status	What to say
GitHub	✅ Completed	“Syncs our port.io repo, users, organization. Powers the CI workflow and Snyk scan action.”
Datadog	✅ Completed	“11 monitors synced. 4 Synthetics health checks showing real OK states.”
PagerDuty	✅ Completed	“6 services, on-call schedules, incident management. All linked to Port service entities.”
Snyk	✅ Completed	“18 projects scanned. 8 real CVEs detected from our npm dependencies.”
Jira	✅ Completed	“14 issues in the KAN project. 6 auto-created by Port automation.”

Key line: > “Five integrations, all healthy, all syncing continuously. This is what Port calls ‘automatic discovery’ — the catalog stays up to date without anyone maintaining it.”

If asked about the GitHub Teams/PR warning: > “The GitHub App is scoped to the port.io repo only — intentionally. We don’t want Port touching our other production repos. The PR data is managed directly in Port.”

STOP 7 — Datadog + Wrap-up (2 min)

Where: Dashboard → Datadog monitors table → click the Datadog dashboard link

What to say: > “Let me show you the observability layer. These are live Datadog Synthetics monitors.”

In the Datadog monitors table: - Point to `[Synthetics] checkout-service health check` → **OK** - Point to `[Synthetics] payment-gateway health check` → **OK** - Point to `[Synthetics] inventory-api health check` → **OK** - Point to `[Synthetics] auth-service health check` → **OK** - “Four services with active health checks. All green.”

Click “Open Datadog Dashboard” link (in the markdown widget): - Opens `https://app.datadoghq.eu/dashboard/trb-bu6-gcs` - Show the 7-day graphs: RPS, P99 latency, error rate, CPU, memory - “7 days of metrics. You can see the traffic patterns — peak hours 9am to 9pm, trough at 2am. This is what the platform team monitors.”

The close: > “So to summarize the flow: developer pushes code → Port scorecard checks all rules instantly → developer triggers Snyk scan from Port → on-call assigned via Port → Gold scorecard achieved → automation creates Jira ticket → ArgoCD aligns K8s → if incident, PagerDuty pages on-call and MTTR is tracked here in Port. Seven steps, fully automated, zero platform team involvement.”

Final line: > “At Walmart scale — 12,000 developers, 10 global hubs — this means the platform team stops being a bottleneck and starts being an enabler. That’s the value of an Internal Developer Portal.”

🚨 THINGS TO AVOID / WATCH OUT FOR

Risk	What to do
Snyk scan takes >2 min	Say “it’s running in GitHub Actions, let me show you the result from a previous run” → go to GitHub Actions tab

Risk	What to do
Jira action fails	Say “let me show you KAN-15 which was created earlier” → go to Jira issues in Port
Datadog dashboard slow to load	Have it open in a separate tab before the demo starts
Port page loads slowly	Pre-navigate to each section before the demo
Asked about “No Data” in Datadog pie	“Those are infrastructure monitors watching for host-level metrics. Since we’re running mock services, there are no real hosts to monitor. The Synthetics monitors — which actively ping endpoints — are all green.”

📋 PRE-DEMO CHECKLIST (do this 10 min before)

- ☐ Open Port.io in browser, logged in as Ohad Biller
- ☐ Navigate to “Walmart - Production Readiness” dashboard — confirm it loads
- ☐ Open Datadog dashboard in a separate tab: `https://app.datadoghq.eu/dashboard/trb-bu6-gcs`
- ☐ Open GitHub repo in a separate tab: `https://github.com/officialbillerin-cloud/port.io`
- ☐ Open Jira KAN board in a separate tab: `https://ohad77.atlassian.net/jira/software/projects/KAN/boards`
- ☐ Have this cheat sheet open on a second screen or phone

🗣 KEY PHRASES TO USE

- **“Single pane of glass”** — when showing the dashboard
- **“Automatic discovery”** — when showing integrations syncing
- **“Gold scorecard = ship it”** — when showing scorecard rules
- **“Zero platform team involvement”** — when showing self-service actions
- **“19 automations running 24/7”** — when showing the automation
- **“Before Port... With Port...”** — use this contrast throughout
- **“At Walmart scale — 12,000 developers”** — to anchor the business context

---

📊 NUMBERS TO REMEMBER

What	Number
Services in Port	6

What	Number
Integrations (all healthy)	5
Self-service actions	9
Scorecards	6
Automations	19
K8s Pods	17
Snyk CVEs (real)	8 (3C, 3H, 2M)
Datadog monitors	11 (4 Synthetics OK)
Jira tickets	14
GitHub PRs	3
ArgoCD apps	6 (all Synced)
Walmart developers	12,000
Walmart global hubs	10
Cost of bad deploy	$2–5M